

# REPA: Reconfigurable PIM for the Joint Acceleration of KV Cache Offloading and Processing

Yang Hong

yang-hong@sjtu.edu.cn  
Shanghai Jiao Tong University  
Shanghai, China

Bo Peng\*

pengbo\_michael@sjtu.edu.cn  
Shanghai Jiao Tong University  
Shanghai, China

Junlong Yang

klearlove@sjtu.edu.cn  
Shanghai Jiao Tong University  
Shanghai, China

Jianguo Yao\*

jianguo.yao@sjtu.edu.cn  
Shanghai Jiao Tong University  
Shanghai, China

## Abstract

The use of KV cache in LLM inference leads to large memory footprint and sub-optimal decoding performance. Prior studies typically address one of these two limitations by either offloading or stage-split inference. In this paper, we explore and reveal the possibility of a joint solution, and propose REPA, a GPU-PIM hybrid system to prototype this idea. We leverage reconfigurable ReRAM PIM to achieve fast KV cache persistence, and balance the requirement of processing speed and memory capacity. To fully unleash the parallelization potential of REPA, we propose optimizations in (1) architecture, (2) data mapping and (3) pipelining: (1) We propose *bulk-wise memory instructions* and *multi-level controllers* to enable finer-grained parallelism in the PIM device. (2) We propose *locality-aware data mapping* to make the best of the aforementioned architectural optimization, and reduce long-range data transfer on chip. (3) We adopt *sub-batch pipelining* to reduce idleness in batches, and propose *transfer overlapping* to shadow the KV cache transfer by computation. Experimental results show that REPA exhibits high inference speed, energy efficiency and integratability. It is 1.5–6.5× faster, and 8–10× more efficient than NVIDIA A100. It also outperforms state-of-the-art DRAM PIM systems by up to 1.4× for long context inference. When integrated into existing offloading systems, REPA achieves 1.4–2.0× offloading speed, and 1.2–1.4× end-to-end speedup, showcasing its high potential for fast KV cache offloading and processing.

**CCS Concepts:** • Computer systems organization → Heterogeneous (hybrid) systems.

\*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, March 22–26, 2026, Pittsburgh, PA, USA.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790212>

**Keywords:** Processing-in-Memory; Large Language Models; KV Cache; ReRAM

## ACM Reference Format:

Yang Hong, Junlong Yang, Bo Peng, and Jianguo Yao. 2026. REPA: Reconfigurable PIM for the Joint Acceleration of KV Cache Offloading and Processing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3779212.3790212>

## 1 Introduction

Transformer-based large language models (LLMs) emerge as powerful tools for natural language processing [5, 9, 10, 12, 71]. As fundamental building blocks, they empower complex AI applications such as conversational systems, image generation, and embodied intelligence [6, 37, 42, 45, 52, 60]. The key to the extraordinary performance of LLMs is multi-head attention [66], a structure predicting output tokens by key-value based context matching and retrieval. Collectively termed the “KV cache”, the keys (K) and values (V) are preserved and reused throughout the entire inference process [48, 57]. This space-for-time trade-off eliminates repetitive computation, which is the very reason making transformer a practical LLM solution. Yet on the other hand, the use of KV cache issues two problems in GPU-based systems:

**Problem #1: memory footprint and offloading overhead.** KV cache accounts for 30%–80% of the overall GPU memory usage in LLM inference [72, 74, 78]. Scaled with the sequence length, it becomes one of the major causes of the gap between memory requirement and provisioning. Offloading is now a de-facto solution to overcome this challenge [34, 51, 59, 61]. However, it incurs significant overhead, especially when the KV cache needs to be persisted to support advanced features like long conversation preservation and fault tolerance [31, 49, 67]. In our early-stage evaluation, we observe a 0.3–2.0× slowdown when 1–4 evictions are triggered in an SSD-based offloading system (see Section 2).

**Problem #2: sub-optimal performance in decoding.** In the decoding stage of inference, output tokens are generated

auto-regressively. GPUs are typically not maximally used in this stage, as KV cache processing has very low arithmetic intensity [15, 19, 53, 55, 79]. Considering the size of the KV cache, GPUs have to frequently move its slices into and out of streaming multiprocessors, resulting in significant data transfer overhead. Moreover, the KV cache cannot be shared across different requests, which means we cannot simply improve the utilization of GPU by batching [53].

Existing studies address the above problems separately. Offloading systems approach problem #1 by finding better eviction and reloading timings [13, 14, 51, 59, 61]. However, for most of these systems, the offloaded KV cache will not be processed until the task is rescheduled. Stage-split inference systems approach problem #2 by executing prefill and decoding on GPUs with different computation capabilities [20, 21, 27, 55]. Although this design improves the utilization of high-performance GPUs, the performance pit in decoding is basically unsolved, but just left to wimpy GPUs.

We observe possibilities of addressing these problems jointly by PIM techniques. From the perspective of hardware characteristics, PIM is both the data storage and acceleration hardware. From the perspective of task features, we notice that all non-batchable operations in decoding are included in KV cache processing. Offloading KV cache to PIM, as a result, leads to both the acceleration of these non-batchable operations, and the possibility of higher GPU utilization. In this paper, we propose REPA, a GPU-PIM hybrid system to prototype this idea. In order to achieve non-volatility, and balance performance and capacity, we choose resistive memory (ReRAM) and reconfigurable computing [2, 4, 23, 32, 68] for our PIM device, REPA-PIM. To justify this decision, we include a detailed feature and applicability analysis, comparing the chosen technique with other popular PIM techniques in Section 3. We also perform comprehensive tests against these solutions in Section 8 to further justify our decision from the perspective of empirical results.

Parallelism is both an opportunity and challenge to our system. On one hand, our PIM technique naturally supports higher parallelism: First, ReRAM cell arrays allow parallel operations on multiple wordlines; second, reconfigurable PIM performs most computation within cell arrays, eliminating the dependence on per-bank CMOS logic. While on the other hand, these features are not fully explored for KV cache processing, which calls for the development of more targeted optimizations, especially in micro-architecture and data mapping. Moreover, parallelization is also a must for REPA, as we need to maximally leverage parallelism to overcome the relatively slower per-operation speed of reconfigurable PIM. In this paper, we propose optimizations in (1) micro-architecture, (2) data mapping and (3) pipelining to better parallelize REPA:

(1) *Micro-architecture* (Section 5). We propose *bulk-wise memory setting instructions* to support wordline parallelism, and reduce the number of instructions issued in computing.

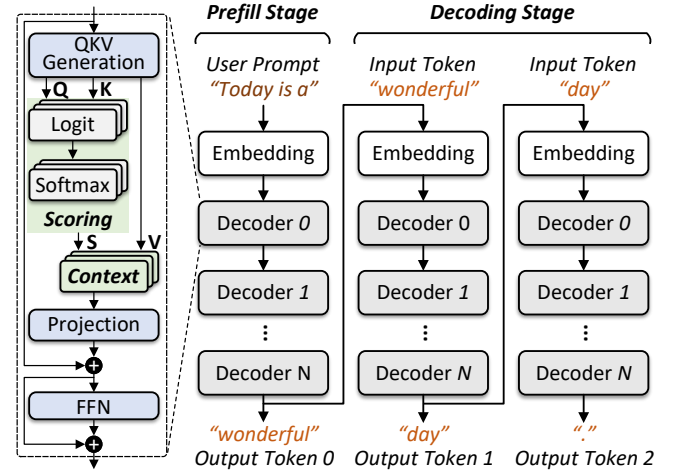


Figure 1. LLM model structure and inference procedure.

We also propose *multi-level controllers*. By placing controllers deeper into the PIM hierarchy, REPA is open for more flexible and finer-grained parallelization control.

(2) *Data mapping* (Section 6). Locality is the key to fully leverage the optimization in micro-architecture. We propose the *locality-aware mapping* strategy slicing KV matrices into larger chunks, and mapping them to nearby cell arrays. This design maximally uses the bulk-wise instructions, and reduces the long-range data transfer in our system.

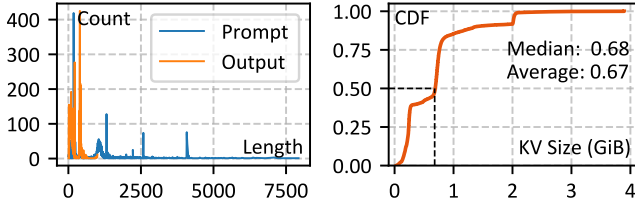
(3) *Pipelining* (Section 7). We improve hardware utilization by the pipelining of GPU and REPA-PIM. We use *sub-batch pipelining* to interleave GPU and PIM operations, and reduce the idleness inside a batch. We also propose *transfer overlapping* to further shadow the KV transfer by computation.

REPA is the first leveraging reconfigurable PIM for the acceleration of KV cache offloading and processing. It is 1.5–6.5× faster and 8–10× more efficient than the GPU-only (NVIDIA A100) system. REPA is at the same time highly integratable. It boosts the offloading in FlexGen by 0.4–1.0×, and leads to 1.2–1.4× and 0.3–0.5× improvement in end-to-end latency and throughput, respectively. This result suggests that our work can be used as a patch to enhance existing systems, which may also envision a large-scale disaggregated system with high-performance interconnect like CXL.

## 2 Background and Motivation

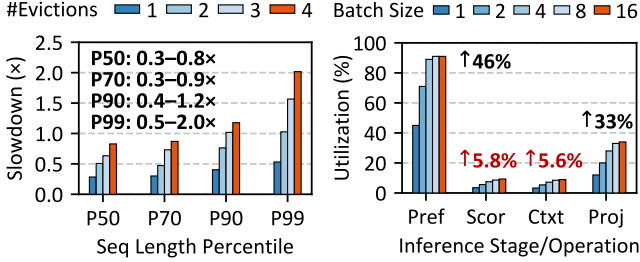
As illustrated in Figure 1, LLM inference comprises two stages. The first is *prefill* summarizing user prompt and generating the first output token. The second is *decoding*, during which output tokens are generated auto-regressively.

Multi-head attention is the core of LLM. It functions by query-based context matching and retrieval. The model firstly encodes the user prompt into the query matrix  $Q$ , and matches it with the key matrix  $K$  by  $\text{logit}(Q \times K^T)$  and softmax. Termed as “scoring”, this process generates a score matrix  $S$ . Each of its row,  $s_i$ , represents a probabilistic mixture of context



(a) Sequence length distribution. (b) CDF of per-request KV size.

**Figure 2.** The sequence length and per-request KV cache size of Azure23 dataset. We use Llama2-7B for this test.



(a) Inference slowdown due to SSD-based KV cache offloading. (b) GPU utilization of inference stages under different batch size.

**Figure 3.** Overhead of KV cache offloading and processing. “Pref” denotes *prefill*. “Scor”, “Ctxt” and “Proj” denotes the *scoring*, *context* and *projection* operation in decoding, respectively.

information matching input token  $q_i$ . Then, the model uses this score matrix to retrieve and blend the context from the the value matrix  $V$  for each input token. This process is performed by  $S \times V$ , and we call it “context”. Once a new output token is generated, we append a new row to  $Q$ ,  $K$  and  $V$ , respectively. When the previous  $K$  and  $V$  matrices are preserved, we only need to use the new query token  $q$  to perform scoring and context, which prevents repeated computation. Here, the  $K$  and  $V$  matrices function as a cache, and thus we collectively call them the “KV cache”.

KV cache is characterized by high memory footprint and low arithmetic intensity. As shown in Figure 2, real-world inference requests have long sequences, and consequently, large KV cache. In the Azure23 dataset [55], the average KV cache size is 670MiB for a 7B model. To quantitatively study how KV cache offloading affects the inference performance, we test the incurred inference slowdown with respect to the sequence length and number of evictions. As shown in Figure 3a, we observe 0.3–0.8 $\times$  and 0.5–2.0 $\times$  slowdown for median-length and P99-percentile requests respectively. This suggests that the inference serving system suffers about 30% performance loss offloading KV cache to the SSD.

In addition, the scoring and context operation in decoding has low arithmetic intensity, which causes low GPU utilization. To quantitatively illustrate this, we test the GPU utilization of different inference stages and operations under various batch size settings. As shown in Figure 3b, GPU cannot be fully utilized for scoring and context, even under large

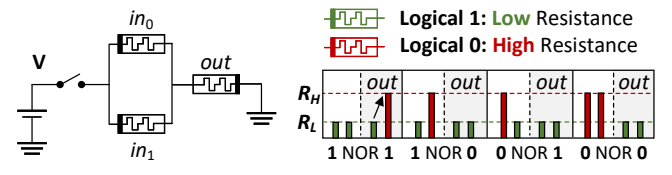
batch sizes. The utilization increases by 5.8% and 5.6% respectively when we increase the batch size to 16. In comparison, we observe a 33% increase for projection. This difference is due to the fact that scoring and context are non-batchable, which is also inevitable when they are processed by the GPU.

### 3 PIM Technique Decision

We choose reconfigurable ReRAM PIM as the PIM technique for REPA. In this section, we introduce how it works (Section 3.1), and justify this design decision by a comparison against three widely-adopted PIM solutions (Section 3.2).

#### 3.1 Reconfigurable PIM with ReRAM

ReRAM is one of the fastest non-volatile memory mediums [63], which is reported to be 100–1000 $\times$  faster than flash [46]. ReRAM is fast in reading, which is comparable to DRAM [8, 69]. Its write performance is relatively lower ( $\sim 5\times$  slower than DRAM with no optimization), but has been successfully improved to 91%–94% of the DRAM performance in recent research [69]. Density is another highlight of ReRAM. It supports the  $4F^2$  density (F is the technode), which enables a denser layout and a potentially higher capacity than DRAM.

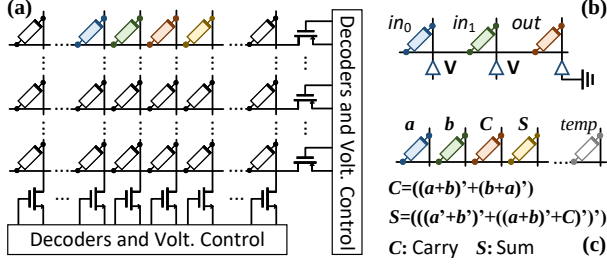


**Figure 4.** NOR gate built from ReRAM cells.

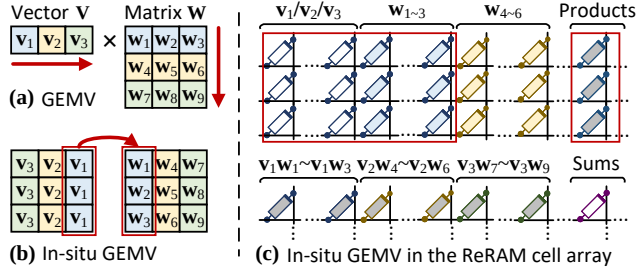
As illustrated in Figure 4, ReRAM cells use the high and low resistance state to represent logical 0 and 1, respectively. State flip requires a strong enough voltage to initiate. When the positive pole (the left side of  $in_0$  and  $in_1$ ) has higher electric potential, the resistance decreases. Similarly, the resistance increases when the potential at the negative pole is higher. Prior work uses this feature to build logic gates out of ReRAM cells [2, 4, 23, 32, 68]. Figure 4 illustrates the implementation of the NOR gate and its truth table.

As shown in Figure 5a and 5b, we can layout such logic gates in the cell array, and construct our desired computational logic. Figure 5c shows the 1-bit addition using ReRAM NOR gates. More complex logic, such as multiplication and floating-point computation has also been explored by prior work [24, 77]. Reconfigurable PIM enables massive parallelizations in memory, which can be used to accelerate tensor operations such as general matrix-vector multiplications (GEMVs). Figure 6 illustrates the computation of GEMV by reconfigurable PIM. Here, the matrix is stored in its transposed form in memory (see Figure 6b and 6c). Since ReRAM does not need pre-charging, we can activate multiple wordlines simultaneously. To perform  $V \times W$ , we parallel  $v_1 \times w_1$ ,  $v_1 \times w_2$  and  $v_1 \times w_3$  on each wordline, and store the partial





**Figure 5.** (a) ReRAM cell array. (b) NOR gate layout in the cell array. (c) In-situ 1-bit addition using NOR.



**Figure 6.** Reconfigurable ReRAM PIM for GEMV.

products. Then, we repeat this process for  $v_2 \times w_4 \sim w_6$ , and  $v_3 \times w_7 \sim w_9$ . After that, we accumulate the partial products on each row in parallel and produce the output vector.

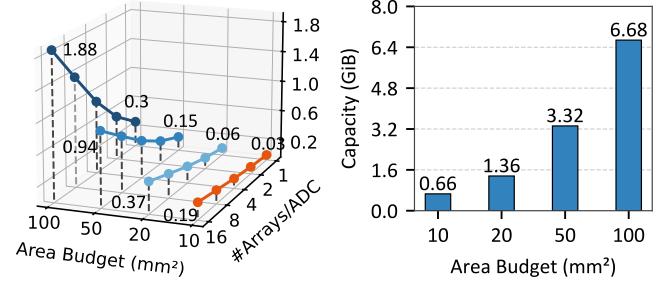
Reconfigurable PIM performs most computation *inside* memory cell arrays. This offers two benefits echoing our requirement for fast KV cache offloading and processing. First, it enables massive in-memory parallelization. Computation will no longer be bounded by the near-bank CMOS logic, leading to a higher theoretical parallelization ability. Second, reconfigurable PIM supports higher memory capacity. The minimized requirement for the peripheral logic significantly reduces the area overhead. Combined with the excellent density of ReRAM cells, it achieves very high capacity, meeting the key requirement of a KV cache offloading system.

**Table 1.** Design choices for REPA-PIM. *The speed is measured by the time required for completing a single operation.*

| Architecture  | Speed  | Scalability | Capacity | Non-volatile |
|---------------|--------|-------------|----------|--------------|
| DRAM PIM      | High   | Medium      | High     | ✗            |
| Reconf. DRAM  | Low    | High        | High     | ✗            |
| Analog ReRAM  | High   | High        | Low      | ✓            |
| Reconf. ReRAM | Medium | High        | High     | ✓            |

### 3.2 Comparison Against Other PIM Solutions

To further justify our design choice, we compare it with three representative DRAM/ReRAM PIM architectures. We summarize their characteristics in Table 1, and discuss their features in detail in the remainder of this section.



**(a)** Analog PIM capacity w.r.t. area budget and ADC density. **(b)** Reconfigurable ReRAM PIM capacity w.r.t. area budget.

**Figure 7.** Capacity of analog vs. reconfigurable ReRAM PIM within 10–100mm<sup>2</sup> area constraints at the 14nm technode.

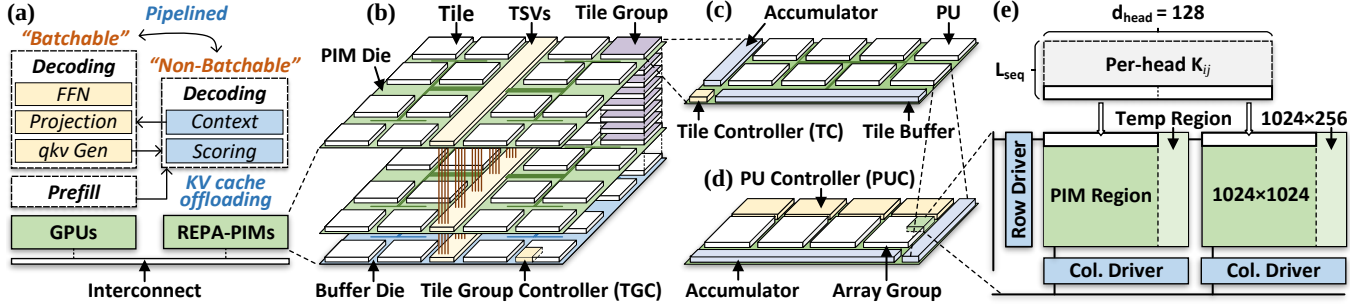
DRAM PIM [15, 18, 19, 53] is a widely-adopted PIM solution based on the high-bandwidth memory architecture [29], using on-die CMOS logic to perform computations. Though faster than reconfigurable PIM in single operations, DRAM PIM has limited potential in massive parallelization, as its computational logic is typically shared across multiple cell arrays or banks [18, 53]. In section 8.3, we show that our solution outperforms state-of-the-art DRAM PIM systems.

Reconfigurable DRAM PIM [16, 41, 79] uses DRAM to build logic gates. However, most existing solutions cannot fully unleash the parallelization potential of reconfigurable computing, which is due to two reasons. First, DRAM cells are volatile and need pre-charging before reading. This makes it difficult to achieve wordline parallelism like ReRAM. Second, many solutions are based on traditional DRAM architectures, where the per-channel memory controller can lower the parallelization degree of the system. In this paper, we overcome these drawbacks by bulk-wise memory instructions and multi-level PIM controllers (Section 5.2).

Analog ReRAM PIM [8, 26, 44, 64, 70] is an architecture with high speed but low memory capacity. It is highly specialized for GEMV acceleration, where computations are performed by the integration of currents. Its bottleneck is the requirement of high-precision ADCs in result decoding. As reported by previous studies, ADCs account for more than 50% of the chip area [50, 70], which consequently reduces the area for memory cell arrays. As illustrated by our quantitative evaluation in Figure 7, analog PIM has 3.6–22× less memory capacity than the reconfigurable solution under the same area budget. This makes it inadequate for the offloading of the memory-hungry KV cache data.

## 4 System Overview

REPA is a GPU-PIM hybrid system. As illustrated in Figure 8a, the system has two types of devices: GPU and REPA-PIM. GPU performs the entire prefill stage, and all batchable tasks



**Figure 8.** (a) REPA System. (b) REPA-PIM architecture. (c)(d)(e) Layout of the tile, processing unit (PU) and cell array.  $d_{head}$  and  $L_{seq}$  in (e) denotes the per-head feature dimension and sequence length.  $K_{ij}$  in (e) denotes the  $K$  matrix of the  $i$ -th head of decoder  $j$ .

in decoding (including qkv generation, projection and feed-forward). REPA-PIM performs all non-batchable tasks (i.e., scoring and context in decoding).

REPA-PIM has a heterogeneous 3D-stacked architecture, and supports fine-grained parallelization (Section 5). We achieve high parallelism by bulk-wise memory setting instructions and the multi-level-controller design. The former enables wordline parallelism inside a cell array, reducing the instructions to be processed. The latter enables more flexible control of such a finer-grained parallelization, trading  $5.76\text{mm}^2$  per-die area overhead for  $3.91\times$  speedup.

The fine-grained parallelization ability of REPA-PIM motivates the locality-aware data mapping (Section 6). We do not interleave per-head KV matrices into far apart “banks” or “channels” like many studies. Instead, we partition them into larger slices, and place these slices onto nearby cell arrays to fully leverage the locality of reconfigurable computing. This fulfills the parallelization potential of bulk-wise instructions, and reduces the on-chip data traffic significantly.

We also develop pipelining techniques to maximally utilize GPU and REPA-PIM (Section 7). Since parallelism in REPA is fine-grained, we do not use the deep head- or decoder-level pipelines. Instead, we use sub-batch pipelining to ensure neither GPU nor REPA-PIM are idle inside a batch. We also propose transfer overlapping, which shadows the transfer of KV matrices and qkv vectors with GPU or PIM computation.

## 5 REPA-PIM Architecture

### 5.1 Memory Layout

As illustrated in Figure 8b, REPA-PIM takes a 3D-stacked architecture design. The device comprises a buffer die and eight PIM dies connected by through-silicon vias (TSVs).

Memories in REPA-PIM are hierarchically organized, which is similar to the HBM. REPA-PIM has 16 tiles on each PIM die. Analogous to the HBM channel, tiles are vertically organized into tile groups enabling full parallelization. The tile performs PIM operations by 8 processing units (PUs). Analogous to the memory bank, a PU comprises 128  $1024\times 2560$  cell arrays divided into 4 array groups. In order to prevent the sneak

**Table 2.** #MemOps/cell of 16-bit fixed-/floating-point (FX/FP) reconfigurable and DRAM PIM. We refer to FloatPIM [24] (reconfigurable) and TransPIM [79] (DRAM) for PIM procedures.

| Computation Type  | Reconf. PIM | DRAM PIM | $\times$ MemOps |
|-------------------|-------------|----------|-----------------|
| FX addition       | 12          | 3        | 4.0             |
| FP addition       | 20          | 3        | 6.7             |
| FX multiplication | 96          | 3        | 32.0            |
| FP multiplication | 23          | 3        | 7.7             |

current affect [30], we implement the cell array by cascading two  $1024\times 1280$  sub-arrays. As shown in Figure 8e, bitlines of these sub-arrays can be independently selected by their own column drivers. This design facilitates the addressing, and improves the parallelism of PIM computation. Each sub-array has a  $1024\times 1024$  PIM region for KV storage and PIM computation, and a  $1024\times 256$  temp region for intermediate data. The reason why we cascade two, rather than more sub-arrays is that the 2048-column PIM region is perfectly suitable for the storage of the per-head KV matrices within a decoder block. As illustrated in Figure 8e, such matrices have an invariant column size, which is the per-head feature dimension,  $d_{head}$ . We notice that  $d_{head} = 128$  for most LLMs. This implies that the storage of a per-head  $k$  or  $v$  vector needs 2048 memory cells under 16-bit data format, which is precisely twice the bitline width of the PIM region.

### 5.2 PIM Control

Since most computation in reconfigurable PIM can be performed by pure memory instructions, we extend the DRAM instruction interface for REPA-PIM. A major challenge here is the massive number of instructions required in computation. As illustrated in Table 2, reconfigurable PIM needs  $4\text{--}32\times$  more operations per cell than DRAM PIM, which becomes a potential source of performance loss. The key idea to address this challenge is parallelism, which we achieve by a joint optimization on instructions and micro-architectures.

**Bulk-wise memory setting.** We propose the bulk-wise memory setting instruction (BLK\_SET) to parallelize NOR-based addition and multiplication on multiple wordlines. As

discussed in previous research, reconfigurable PIM performs addition and multiplication by a sequence of in-situ NORs [1, 24]. Recall Figure 4 in Section 3.1, such NORs are conceptually memory settings activating two input cells, and setting the output cell with the current generated from the inputs. Therefore, to perform a specific bulk setting, we need to specify: (1) wordlines to be parallelized, (2) two bitlines for input, and (3) the bitline for output.

**Table 3.** Format of the BLK\_SET instruction.

(a) Overview of BLK\_SET.

| Field | Opcode | Block Addr. | Input1 | Input2 |
|-------|--------|-------------|--------|--------|
| #Bits | 8      | 24          | 16     | 16     |

(b) Format of the block address in BLK\_SET.

| Field | Rsv. | TG | Tile | PU | AG | Arr. | Block |
|-------|------|----|------|----|----|------|-------|
| #Bits | 3    | 4  | 3    | 3  | 2  | 5    | 4     |

Since types of sub-NORs inside an addition or multiplication are fixed, REPA-PIM can infer the bitline of the output cell with the NOR type. Taking fixed-point multiplication as an example, the computation contains 3 types of NORs for partial product and 11 types of NORs for addition, each of which has fixed output offset relative to the input [1]. We also notice that for a specific multiplication or addition, the output cells of previous NORs/memsets are not reused as the outputs of their successors. This means we can acquire the offset by the NOR type, and infer the output bitline by offsetting from the beginning of the temp region. Through this design, the instruction length is constrained to 64 bits, and we only need to specify three operands listed as follows:

(1) *Block address.* BLK\_SET specifies a group of 64 adjacent wordlines for parallel memory settings. Named “memory block” in REPA-PIM, such a memory region is identified by a 24-bit address illustrated in Table 3b. The higher 3 bits are reserved bits, and the remaining bits help identify the block location through the tile group (TG), tile, PU, array group (AG) and block hierarchy. A question is that why we do not specify the memset range by specific wordlines. The reason for this design decision is two-fold. First, it eliminates variable-length parameters resulted from the per-wordline range specification, which lowers the complexity of instruction decoding. Second, by using memory block addresses, our strategy simplifies the addressing mechanism, reducing both address storage and translation overhead.

(2) *Two input bitlines.* The *Input1* and *Input2* operands specify the input bitlines for an in-memory NOR. Each of them have 16 bits, supporting up to  $2^{16}$  columns of a cell array. REPA-PIM has 2560 columns per array, thus 13 out of 16 bits are used to address each input bitline.

**Multi-level controllers.** As shown in Figure 8, we use *tile group*, *tile* and *PU controllers* to enable fine-grained parallelism in REPA-PIM. The tile group controller (TGC) is

**Table 4.** Speedup and per-die area overhead w.r.t. #controllers/PU. We test the *logit* ( $\mathbf{q} \times \mathbf{K}^T$ ) operation, with the area overhead estimated at the 14nm technode.

| #Controllers/PU                 | 1    | 2    | 4    | 8     | 16    | 32    |
|---------------------------------|------|------|------|-------|-------|-------|
| ×Speedup                        | 1    | 1.95 | 3.91 | 5.20  | 7.17  | 9.83  |
| Per-die Area (mm <sup>2</sup> ) | 1.92 | 3.84 | 7.68 | 15.36 | 30.72 | 61.44 |
| ΔSpeedup/ΔArea                  | -    | 0.49 | 0.51 | 0.16  | 0.13  | 0.09  |

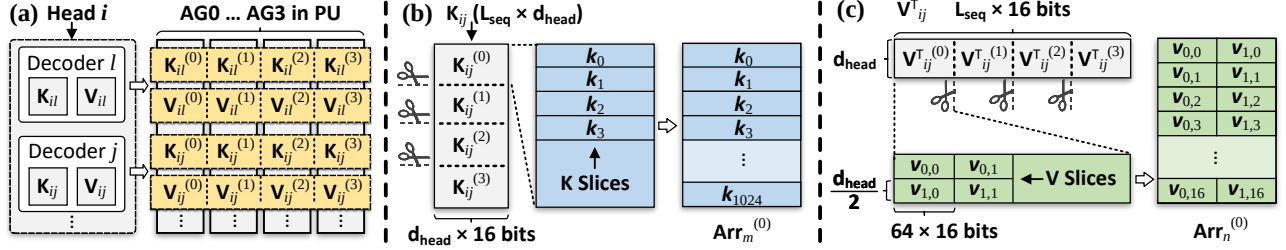
analogous to the per-channel HBM controller. The difference is that tile group controllers do not directly manipulate ReRAM arrays. Instead, they dispatch PIM operations to designated tile controllers, which parallelizes all their tiles. The tile controller (TC) parallelizes its PUs by forwarding the dispatched operations. It also controls the accumulation of partial results produced by each PU. The PU controller (PUC) is an extension of cell array drivers, which is responsible for the manipulation of ReRAM cells. We place four such controllers in one PU, each parallelizing a specific group of cell arrays. We do not arrange more controllers in PU, as the four-controller setting is already sufficient for good parallelism. A per-head K/V matrix requires 1MiB memory space under the maximum 4096 sequence length, which is the PIM region capacity of four arrays. This means computation on these arrays can be parallelized if we evenly distribute them to four array groups, and manage their computation by dedicated controllers. As to inner-array parallelism, we leave it to bulk-wise instructions and the independent column drivers of sub-arrays (see Figure 8e).

Another notable fact is that the 4 controllers/PU setting is cost-effective. As illustrated in Table 4, the speedup of  $\mathbf{q} \times \mathbf{K}^T$  scales with #controllers/PU when it is  $\leq 4$ , and the trend significantly slows down when we attempt to arrange more controllers. Compare to the 1 controller/PU setting, we trade a 5.76mm<sup>2</sup> per-die area overhead for a 3.91× speedup. In comparison, increasing #controllers/PU to 32 only attributes to another 2.51× speedup. This is because when we arrange more than 4 controllers for a PU, per-head KV matrices are scattered across many array groups, which increases the data gathering overhead. Moreover, the per-die area overhead of 8, 16 and 32 controllers are too costly for our system, which prevents us from using these settings.

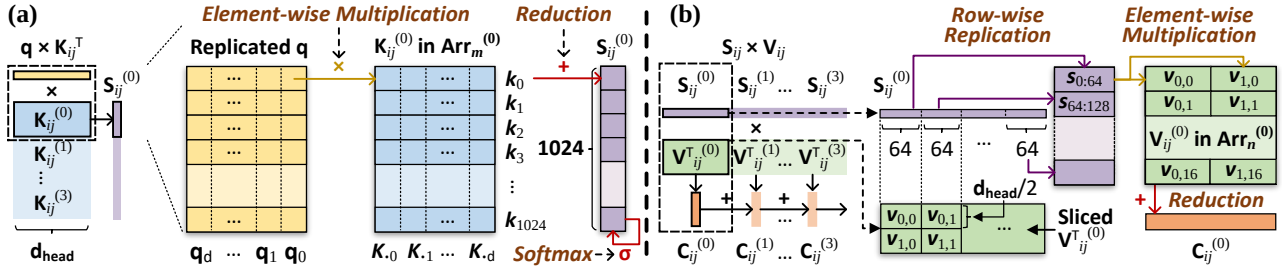
## 6 Locality-aware Data Mapping

### 6.1 Mapping of New Requests

The KV cache of a specific inference request is split into groups by attention head, each of which contains the per-head KV matrices from all decoders. An example of such a KV cache group is shown in Figure 9a, in which we depict the per-head KV matrices of decoder  $j$  and  $l$ . To map the KV cache groups, we sort them by head IDs and place them one after another onto dedicated tile groups with the largest free space. When encountering insufficient resources, we perform



**Figure 9.** (a) Mapping of the per-head KV matrix group. (b)(c) Slicing and mapping of the per-head K and V matrices. “AG” denotes the array group. “ $Arr_m^{(g)}$ ” denotes the cell array  $m$  in array group  $g$ .  $k_0, \dots, k_{1024}$  are row vectors in  $K_{ij}^{(0)}$ .  $v_{r,c}$  are slices of  $V_{ij}^{(0)}$ , where  $r$  and  $c$  represents the row and column index, respectively. Each  $v_{r,c}$  has  $\frac{d_{head}}{2} \times 64$  16-bit values.



**Figure 10.** Reconfigurable PIM implementation of the partial (a) scoring and (b) context operation.  $q_0, \dots, q_d$  are components of the  $q$  vector.  $s_{u:v}$  denotes replicated  $[u, v]$  slices of  $S_{ij}$ , each of which has  $\frac{d_{head}}{2}$  rows (replicas).

scale-out, and leave the unused space for the decode-time KV appending of those already-mapped heads. This strategy maximally parallelizes all tile group controllers, which speeds up the persistence of KV cache. Moreover, splitting the KV cache by attention head incurs very limited overhead. This is because REPA-PIM only needs to gather  $N_{head}$  vector slices (with the size  $1 \times d_{head}$ ) at the end of each decoder block, and broadcast the concatenated  $1 \times (d_{head} \cdot N_{head})$  vector to all tile groups. The fragmentation overhead incurred by scale-out is also moderate, which we show in Section 8.6.

For the per-head matrices in a specific KV cache group (e.g.,  $K_{ij}$ ,  $V_{ij}$ ,  $K_{il}$  and  $V_{il}$  in Figure 9a), the strategy prioritizes free PUs for their placement, enforcing three policies:

(1) *Each per-head matrix is split and placed onto four free cell arrays, each of which belongs to a dedicated array group.* This is because any per-head matrix is 1MiB at most, which is precisely the data capacity of four cell arrays. This split-and-placement strategy offers significant benefits: First, it quadruples the speed of logit and context operations by fully parallelizing all four array groups of the PU. Second, by filling the cell array with data from the same per-head matrix, this strategy fully utilizes the bulk-wise instruction in the upcoming scoring and context computation. Third, it facilitates the gathering of partial results by echoing the high locality of reconfigurable PIM.

(2) *The per-head K and V matrix slices of a specific decoder block (e.g.,  $K_{ij}^{(0)}$ ,  $V_{ij}^{(0)}$  and  $K_{ij}^{(1)}$ ,  $V_{ij}^{(1)}$ ) must be mapped to the*

*same array group.* This is another design echoing the locality of reconfigurable computing. We notice that the partial result of the  $q \times K_{ij}^{(0)}$  is used by  $V_{ij}^{(0)}$ . When placing these slices nearby, we restrict the frequent partial result propagation inside the PU, which significantly reduces the data transfer on external interconnects.

(3) *Per-head KV slices from different decoders are placed sequentially in the array group.* This design eliminates the performance bottleneck in array groups. The array group may become a bottleneck, as the 32 cell arrays in it are managed by a single PU controller. By sequentially mapping KV slices from different decoders to these arrays, we eliminate the need to parallelize the per-group arrays, because at any moment, there will be only one decoder being processed.

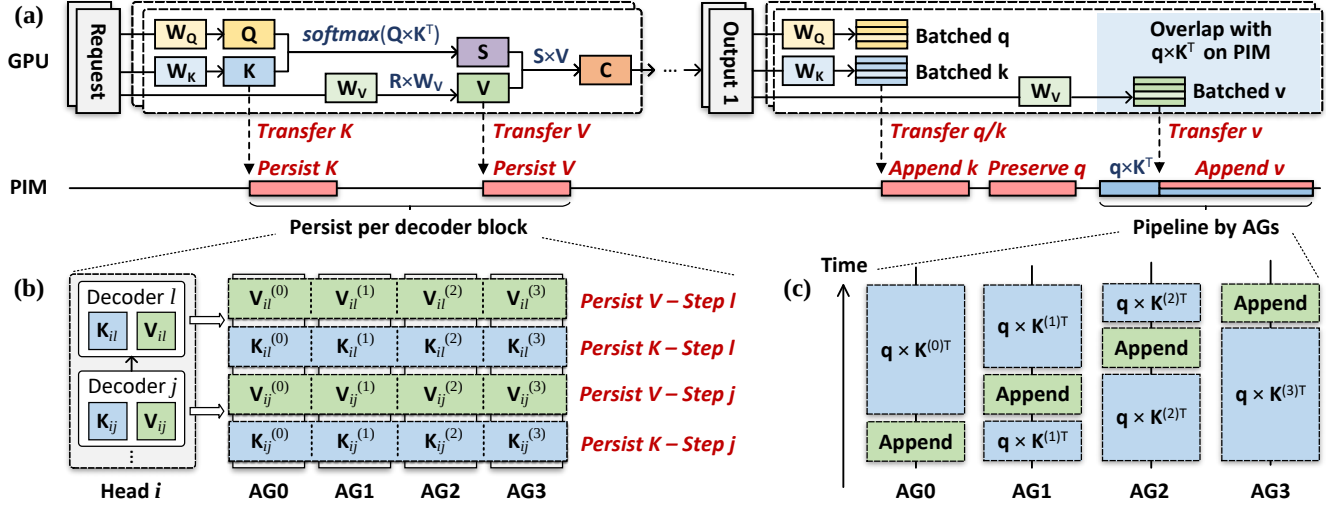
## 6.2 Mapping of the Per-head Matrix

Now, we detail how REPA-PIM slices a specific per-head K/V matrix, how it maps the slices to cell arrays, and how computation is parallelized on these arrays.

**Per-head K mapping.** As illustrated in Figure 9b, we split the per-head K matrix in rows into four slices, and map each slice to a free cell array. For a specific slice (e.g.,  $K_{ij}^{(0)}$  in Figure 9b), we further split it into row vectors, and map each of these vectors sequentially to a row of the cell array.

To perform the  $q \times K_{ij}^T$  operation, we replicate the  $q$  vectors, and parallelize the dot-product on each row of the cell array. As illustrated in Figure 10a, the dot-product is performed by





**Figure 11.** (a) Overlapping of computation and KV transfer. (b) Persistence of prefill KV matrices. (c) Pipelining of  $v$  vector transfer and  $q \times K^T$  computation. Matrix  $R$  in (a) denotes the input requests.

element-wise multiplications (i.e.,  $q_0 \times k_{0,0}, q_1 \times k_{1,1}, \dots$ ) and reductions. The result on row  $r$  is the  $r$ -th component of the partial result. The final activation  $S_{ij}$  can be computed by applying concatenation and softmax on the partial results (i.e.,  $S_{ij}^{(0)} \sim S_{ij}^{(3)}$ ). In this procedure, most computation is performed locally inside array groups, and only the construction of the final result needs inter-group data transfer.

**Per-head  $V$  mapping.** As illustrated in Figure 9c, we map the per-head  $V$  matrix by vertically partitioning its transposed form. For a specific partition (e.g.,  $V_{ij}^{T(0)}$  in Figure 9c), we firstly cut it into two by rows, and then split each of them vertically into slices with  $d_{head}/2 \times 64$  elements. Since the PIM region of each cell array has 2048 columns, we can fit these slices into the array with the layout in Figure 9c.

As illustrated in Figure 10b, the  $S_{ij} \times V_{ij}$  operation is decomposed into four parts. Taking the first part,  $S_{ij}^{(0)} \times V_{ij}^{(0)}$  as an example, it comprises dot-products between several  $S$  and  $V$  slices. In specific, the  $S_{ij}^{(0)}$  vector is split into several 64-element slices. Each slice is then replicated in rows, enabling the parallelism of the dot-products. For example, we can parallelize dot-products between slice  $s_{0:64}, v_{0,0}$  and  $s_{0:64}, v_{1,0}$ . The partial dot-products will be reduced to construct the partial context  $C_{ij}^{(0)}$ . The final per-head context,  $C_{ij}$ , can be constructed by reducing all partial context vectors.

## 7 Pipelining GPU and REPA-PIM

The stage-split inference pattern of REPA has made device utilization a key concern. To maximally utilize GPU and REPA-PIM, we pipeline them by two techniques.

**Sub-batch Pipelining.** Sub-batch Pipelining is an effective idea maximizing device utilization within a hybrid system [19, 53]. In this paper, we interleave requests to GPU and REPA-PIM by their execution stage. Here, a batch is split

into two sub-batches. REPA executes them alternately on GPU and REPA-PIM to keep both of them busy. The size of sub-batches are adaptively decided by the computation capability of the device, which we define by the per-workload performance within a recent time period. Given that most operations in inference are matrix-vector or matrix-matrix multiplications, REPA uses the scale of such operations (i.e., number of scalar multiplication and addition) to estimate the workload scale. We also incorporate the idea of iteration-level scheduling [76] to allow new and terminated requests to be continuously appended into or removed from the batch.

**Transfer Overlapping.** The size of the KV cache makes its transfer a performance concern. In this paper, we alleviate this overhead by taking the opportunity of overlapping.

**Transfer overlapping in prefill.** As illustrated in Figure 11a, we overlap the transfer of KV matrices with GPU computation. Specifically, we overlap the transfer of  $K$  matrices with scoring and  $V$  generation. After  $V$  matrices are generated, we overlap its transfer with the context operation. As shown in Figure 11b, the prefill KV matrices are transferred and persisted per decoder block. Matrices from different attention heads are persisted in parallel by designated tile groups. During the transfer of per-decoder KV matrices, the computation on other PUs and tiles will not be affected. This high parallelism is attributed to the locality-aware mapping strategy—if we interleaved and mapped per-decoder KV matrices across multiple PUs and tiles, their transfer and persistence would impact the computation of many other requests.

**Transfer overlapping in decoding.** We also overlap the transfer of batched  $q, k$  and  $v$  vectors in decoding. As illustrated in Figure 11a, we overlap the transfer of batched  $q$  and  $k$  vectors with the generation of  $v$  vectors, and overlap the transfer of  $v$  vectors with the  $q \times K^T$  operation on REPA-PIM.



As illustrated in Figure 11c,  $v$  vectors are appended to  $V$  matrices in pipeline. The reason for this design is that  $v$  vectors must be transferred to the PU via the external interconnect. Pipelining  $v$  vector storage by array groups reduces chances of conjunction. For a specific PU, there will be only one array group performing  $v$  vector persistence, and the  $q \times K^T$  computation of other array groups will not be blocked.

## 8 Evaluation

### 8.1 Prototype Implementation

We implement a prototype of REPA in 1.3K and 7.0K LoC respectively in Python and C++.

**Device access and management.** We extend PyTorch [54], and provide a Python wrapper for inference tasks to use REPA-PIM. We also build a runtime system for the management of task status and PIM memory. The PIM memory is addressed in bytes and allocated in  $64 \times 2560$  blocks. As discussed in Section 5, 2560 is the column size of the REPA-PIM cell array, which includes 2048 cells for KV cache preservation and PIM computation, and 512 cells for the temporary storage of intermediate data. We let the number of rows per allocation be 64, as it is a balanced option for efficiency and memory space utilization. Considering the size of the KV cache, the block size per allocation can be larger, which prevents frequent allocation, and reduces the size of the network-memory mapping structure. On the other hand, eschewing an over-sized memory block (e.g.,  $128 \times 2560$  or  $256 \times 2560$ ) helps improve memory utilization, as we have more chances of waste when provisioning larger blocks.

**Reconfigurable computation.** We use FP16 as the data format of REPA, and the solution proposed by FloatPIM [24] for reconfigurable addition and multiplication. For the maximum operation in softmax, we use the fast in-situ maximum solution proposed by ReSQM [38]. REPA needs only 16 cycles to retrieve the maximal from  $N$  FP16 values when they are stored in the same cell array. To calculate the  $e^x$  in softmax, we rewrite it by  $2^{x \log_2 e}$ , and decompose it to a  $2^x$  and a multiplication. The multiplication is introduced earlier in this paragraph, and the  $2^x$  is performed by left-shifting.

### 8.2 Evaluation Setup

**Experimental environment.** We test REPA on an Ubuntu 22.04 server with 40 CPU cores and 8 NVIDIA A100 GPUs. We faithfully implement the design of REPA-PIM into an in-house simulator. The simulator is built over NeuroSim-3D [56], the 3D-stacked version of the time-tested cycle accurate NeuroSim simulator [7]. REPA-PIM operates under 1GHz. It uses a 14nm bipolar resistive technology node, with its parameters set following the VTEAM model [33]. To align with existing work and practical devices [24, 32], we set the switching delay of ReRAM cells to 1ns, and the voltage pulse of SET and RESET to 1V and 2V, respectively. We estimate instruction dispatch latencies by the time-to-flight latency

in controllers and the on-wire transfer latency. The time-to-flight latency is  $\sim 20$ ns. The on-wire latency for TGC $\rightarrow$ TC, TC $\rightarrow$ PUC and PUC $\rightarrow$ DRV is set to 4ns, 2ns and 2ns, respectively. The per-TG/Tile/PU bandwidth is set to 256GB/s, 32GB/s and 4GB/s, respectively.

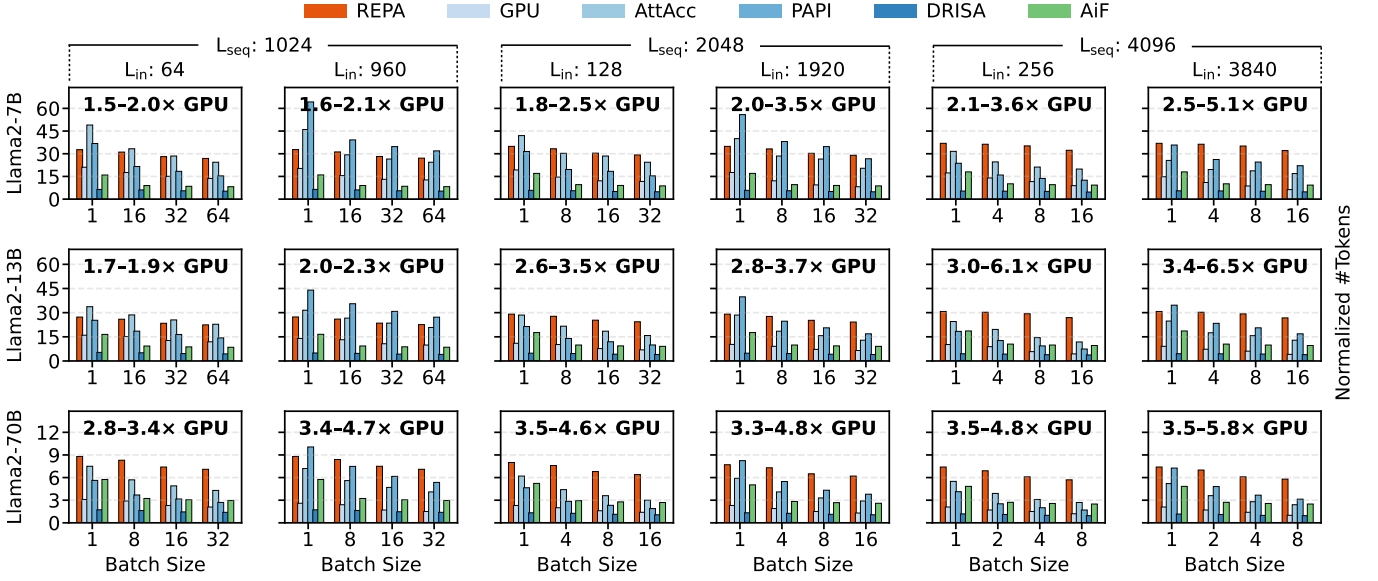
**Experiment design.** We use the 7B, 13B and 70B version of Llama2 [65] as our benchmark, and evaluate REPA by five groups of tests. The first test group focuses on the performance of REPA (Section 8.3): (1) We evaluate the token generation performance of REPA in different sequence length settings to show its potential in inference acceleration. (2) We test how REPA-PIM works well with existing offloading systems (FlexGen [61] in this paper) to show its integrability and end-to-end offloading acceleration ability. The second test group focuses on the justification of key design decisions we made (Section 8.4): (1) We evaluate our architectural design, and answer questions such as why we do not use pure PIM for inference and why the traditional prefill-decoding separation technique [20, 21, 27, 55, 59] is not used. (2) We evaluate the data mapping strategy, and show why locality is important for REPA-PIM. The third test group evaluates the efficiency of REPA-PIM, and shows its superior speed from the aspect of per-energy performance (Section 8.5). The fourth test group evaluates the memory fragmentation during inference across models of different scales (Section 8.6). The fifth test group reports the power consumption and area overhead of REPA-PIM (Section 8.7).

### 8.3 REPA Performance

**Token Generation Ability.** We compare the token generation performance of REPA against five baselines:

- **GPU**, the hardware baseline using NVIDIA A100.
- **AttAcc** [53], a state-of-the-art DRAM PIM solution using near-bank logic for attention acceleration.
- **PAPI** [18], a state-of-the-art DRAM PIM solution considering the dynamic dispatching of compute- and memory-bound tasks to suitable devices.
- **DRISA** [41], a reconfigurable DRAM PIM solution using DRAM cells for in-situ computing.
- **AiF** [35], a state-of-the-art in-flash processing system for LLM inference acceleration.

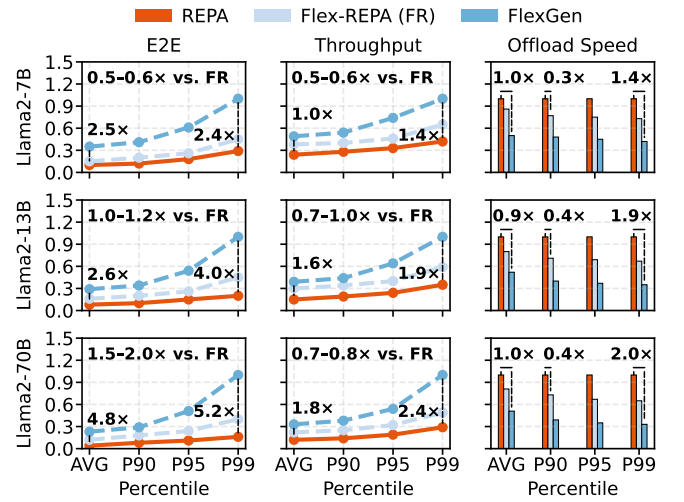
As shown in Figure 12, REPA exhibits superior performance for long sequence, batched requests and larger models. It generates 1.8–4.8 $\times$  and 2.1–6.5 $\times$  more tokens than GPU for the 2048 and 4096 sequence length, respectively. In comparison, the improvement is 1.5–4.7 $\times$  for the 1024 sequence length. Similar results can also be observed when we compare the performance of REPA with AttAcc. For the 4096 sequence length, REPA generates 0.4–1.4 $\times$  more tokens than AttAcc, while the improvement shrinks to -0.3–0.8 $\times$  when the sequence length is 1024. The reason for this result is that REPA has slower per-operation speed but higher



**Figure 12.** Token generation performance of REPA and its baselines. We conduct the test with three sequence length settings (namely 1024, 2048 and 4096), each of which contains two sub-settings representing the short and long input, respectively. For each setting, we collect the number of generated tokens under four batch sizes, and normalize the results per second and request.

parallelization ability. As mentioned in Section 3.2 and 5.2, reconfigurable PIM requires more cycles for a single operation and is thus slower when inadequately parallelized. Given the high single-operation speed of the traditional DRAM PIM, it is not surprising that REPA has inferior performance for one-shot inferences. Once we increase the batch size, it regains its edge as the device is fully parallelized. REPA has better performance than PAPI under larger batch sizes and long sequences. It generates 0.5–1.2 $\times$  more tokens for the 4096 sequence length and the largest batch size settings. However, the performance result varies under different input lengths. For PAPI, the number of generated tokens drops when the sequence is dominant by the output (see results when  $L_{in} = 64, 128$  and 256). This is because PAPI uses the Attn-PIM to process the GEMVs in scoring and context, where each processing unit is shared by two memory banks. This works for short output sequences. However, when the  $L_{out}$ , model size and batch size are all large, these processing units can be overwhelmed, leading to a performance loss.

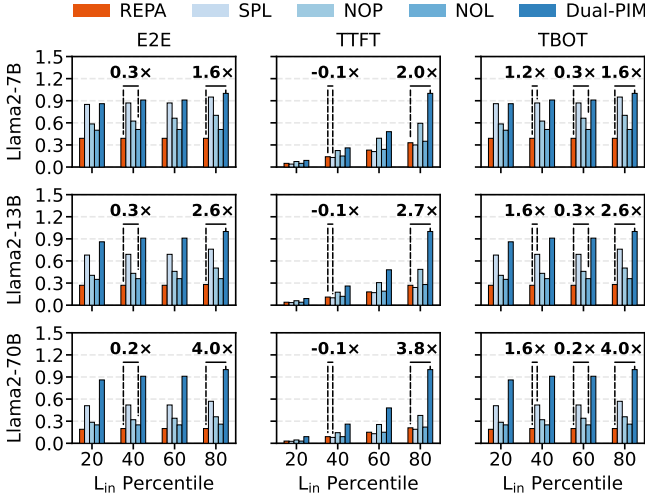
REPA consistently outperforms DRISA (4.1–6.2 $\times$ ) and AiF (0.5–2.6 $\times$ ) under all test settings. The result against DRISA highlights the value of the bulk-wise memory setting instruction. Though slower than DRAM in single-cell update, ReRAM is open for multi-wordline activation, and is thus faster when memory cells are bulk-wisely updated for in-situ NORs. The bulk-wise memory setting instruction makes the best of this feature, which attributes to higher parallelism and superior token generation ability. The result against AiF proves the potential of ReRAM PIM as a solution with both non-volatility and high token generation ability.



**Figure 13.** Normalized end-to-end inference latency, throughput and offloading speed of REPA, vanilla FlexGen and REPA-PIM enhanced FlexGen.

**End-to-end Offloading Ability.** In this test, we evaluate how REPA works well with existing offloading systems. We use FlexGen [61] as the testbed, and show how REPA improves its performance by offloading weights and the KV cache to REPA-PIM (Flex-REPA). We let the offloaded weights and KV cache be processed by REPA-PIM, and leave the offloading strategy unchanged. Since 4-bit quantization is used in FlexGen, we use this setting in REPA and Flex-REPA.

We test the end-to-end latency (E2E), throughput and offloading speed of the three systems using traces from the



**Figure 14.** Normalized E2E, TTFT and TBOT of REPA and its variants. The  $L_{in}$  percentile denotes the proportion of the input length relative to the overall 4096 sequence length.

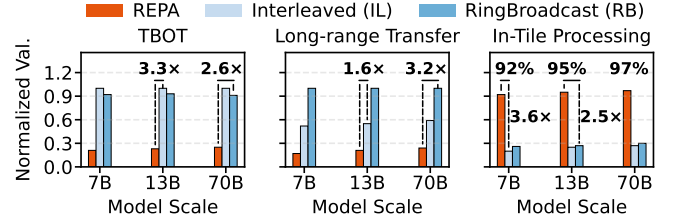
Azure23 dataset [55]. As shown in Figure 13, REPA and Flex-REPA outperform the vanilla FlexGen systems on all metrics and model scales. We observe 2.4–5.2 $\times$ , 1.0–2.4 $\times$  and 0.9–2.0 $\times$  improvement on E2E, throughput and offloading speed, respectively. Compared to the throughput performance, the improvement on E2E is more significant, as FlexGen is specialized for achieving good throughput, and incorporating REPA-PIM helps it dealing with its weakness in latency.

#### 8.4 Design Decision Exploration

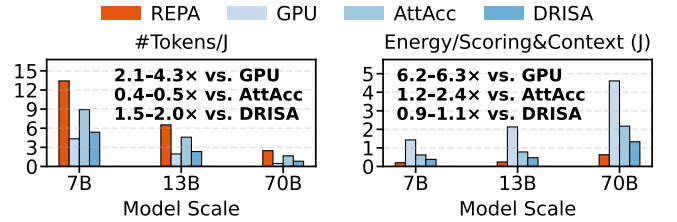
**Evaluation on Architectural Designs.** We include four REPA variants to test the efficacy of our architectural designs:

- **REPA-SPL**, a stage-split variant performing prefill by the GPU, and decoding by REPA-PIM.
- **REPA-NOP**, a variant that does not use sub-batch pipelining for inference.
- **REPA-NOL**, a variant that does not overlap the transfer of KV matrices and vectors with computation.
- **Dual-PIM**, a variant using REPA-PIM for compute-intensive operations.

As illustrated in Figure 14, REPA outperforms all these variants on the E2E latency and time-between-output-tokens (TBOT) metric. The 1.2–1.6 $\times$  improvement over REPA-SPL suggests the effectiveness of performing batched decoding-time projection and FFN by GPU. The 0.5–0.8 $\times$  improvement over REPA-NOP suggests the necessity of performing sub-batch pipelining. The 0.2–0.3 $\times$  improvement over REPA-NOL showcases the effectiveness of transfer overlapping. The 1.6–4.0 $\times$  improvement over Dual-PIM justifies the hybrid architecture taken by REPA. We also notice that REPA is 0.1 $\times$  slower than REPA-SPL on time-to-first-token (TTFT). This is because REPA-SPL does not process the projection and FFN in decoding, which leads to higher prefill performance.



**Figure 15.** Normalized TBOT, long-range data transfer and percentage of in-tile processing of three mapping strategies. “Long-range data transfer” refers to that across different tiles.



**Figure 16.** Energy efficiency on two metrics. The first is the number of output tokens normalized per joule. The second is the energy normalized per scoring and context operation.

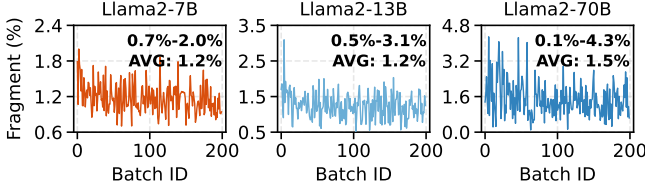
Given that the E2E latency is dominated by the decoding performance, this minor overhead in prefill is negligible.

**Evaluation on Data Mapping.** The data mapping strategy we propose in REPA highlights locality to fully parallelize the computation. To illustrate its performance, we compare it against two widely-adopted ideas in DRAM PIM. The first is *interleaved mapping* (IL). Data is delicately sliced and scattered across channels, banks and even cell arrays to fully leverage the near-bank computational logic. In this test, we construct the IL mapping strategy by generalizing the idea in AttAcc [53]. The second baseline we take is *RingBroadcast* (RB) generalized from TransPIM [79]. This strategy reduces the data gathering overhead by propagating partial results across memory banks. As illustrated in Figure 15, REPA has superior performance on all metrics and model scales. Compared to the IL and RB strategy, it has 3.3 $\times$  and 2.6 $\times$  better TBOT, and has 1.6 $\times$  and 3.2 $\times$  less long-range data transfer. Due to the REPA-PIM architecture, more than 92% of computation is conducted within a single tile, which is 3.6 $\times$  and 2.5 $\times$  better than IL and RB respectively.

#### 8.5 Energy Efficiency

We test the energy efficiency of REPA and three baselines on the 7B, 13B and 70B version of Llama2. As illustrated in Figure 16, REPA generates 2.1–4.3 $\times$ , 0.4–0.5 $\times$  and 1.5–2.0 $\times$  more tokens per joule than GPU, AttAcc and DRISA, respectively. For the efficiency of scoring and context operations, its improvement over GPU and AttAcc expands to 6.2–6.3 $\times$  and 1.2–2.4 $\times$ , respectively. In contrast, its lead over DRISA drops to 0.9–1.1 $\times$ . This is because DRISA uses reconfigurable





**Figure 17.** Memory fragmentation on Llama2-7B, 13B and 70B. We use the Azure23 dataset, and set the batch size to 16.

**Table 5.** Area and power overhead by chip components.

| Comp.                    | Area (mm <sup>2</sup> ) | Power (mW)    | Params.     | Spec.         |
|--------------------------|-------------------------|---------------|-------------|---------------|
| <b>Array Overhead</b>    |                         |               |             |               |
| Cells                    | 0.0016                  | 1.14          | Size        | 1024×2560     |
| SA                       | 0.0005                  | 1.65          | -           | -             |
| DRV                      | 0.0011                  | 0.53          | -           | -             |
| <b>Total</b>             | <b>0.0037</b>           | <b>3.32</b>   | <b>Size</b> | <b>256KiB</b> |
| <b>PU Overhead</b>       |                         |               |             |               |
| Array                    | 0.4136                  | 51.12         | Total       | 128           |
| Acc.                     | 0.0001                  | 0.04          | -           | -             |
| Buffer                   | 0.0034                  | 2.89          | -           | -             |
| Bus                      | 0.0009                  | 2.70          | -           | -             |
| Ctrl.                    | 0.0600                  | 4.80          | Total       | 4             |
| <b>Total</b>             | <b>0.4779</b>           | <b>61.55</b>  | <b>Size</b> | <b>32MiB</b>  |
| <b>Tile Overhead</b>     |                         |               |             |               |
| PU                       | 3.8232                  | 492.41        | Total       | 8             |
| Acc.                     | 0.0001                  | 0.04          | -           | -             |
| Buffer                   | 0.0270                  | 14.17         | -           | -             |
| HTree                    | 0.6590                  | 14.77         | -           | -             |
| Ctrl.                    | 0.0150                  | 1.20          | -           | -             |
| <b>Total</b>             | <b>4.5243</b>           | <b>522.59</b> | <b>Size</b> | <b>256MiB</b> |
| <b>REPA-PIM Overhead</b> |                         |               |             |               |
| <b>Total</b>             | <b>73.2631</b>          | <b>68.92K</b> | <b>Size</b> | <b>32GiB</b>  |

computing throughout the entire inference process, which is slower and less efficient for batchable operations. While for non-batchable scoring and context, both REPA and DRISA use reconfigurable computing. REPA’s improvement in this part is mainly attributed to the energy efficiency of ReRAM.

## 8.6 Memory Fragmentation

We evaluate the dynamic memory fragmentation of REPA using the real-world Azure23 dataset [55]. We use batch size 16 for all tested models, and record the dynamic fragmentation in Figure 17. REPA incurs limited fragmentation ranging from 0.1%–4.3%. This is attributed to the 64×2560 block we use, which prevents over-provisioning of the resource. We also notice that larger models (e.g., Llama2-70B) exhibit higher fragmentation ratio (up to 4.3%). This is because scale-out occurs more frequently as new requests arrive. While the unused memory will be gradually consumed during decoding, fragmentation may still persist, especially when the appended KV vectors cannot fully occupy the available space.

## 8.7 Overhead Evaluation

We summarize the area and power overhead of REPA-PIM in Table 2. The device has a 73.3mm<sup>2</sup> per-die area, in which 26.2mm<sup>2</sup> is arranged for ReRAM cells. The area for the peripheral computational logic is minimized. It is 0.0001mm<sup>2</sup> for each PU and tile, which accounts for less than 0.02% of the overall area overhead. The PU and tile controller also incurs limited area overhead. It is 7.92mm<sup>2</sup> for each die, which is 10.8% of the overall area overhead. As discussed in Section 5.2, we trade this 10.8% additional area for 3.91× speedup. REPA-PIM requires 68.92W power to achieve full parallelism, which is significantly lower than server GPUs.

## 9 Discussion

Endurance is a potential issue of ReRAM that has been recognized and discussed for long [3, 28, 62]. REPA does not suffer this pitfall when high-endurance ReRAMs are used. To prove this, we estimate the number of memsets performed on each ReRAM cell per year. Since REPA offloads the scoring and context computations in attention, which are conceptually GEMVs, we can then estimate using Equation (1). Here,  $N_{\text{memsets}}$  denotes the overall memsets on one ReRAM cell.  $N_{\text{memsets}/\text{GEMV}}$  and  $N_{\text{secs}/\text{year}}$  is the number of memsets per GEMV, and the number of GEMVs per second, respectively.  $N_{\text{secs}/\text{year}}$  is the seconds per year.

$$N_{\text{memsets}} = N_{\text{memsets}/\text{GEMV}} \times N_{\text{GEMVs}/\text{sec}} \times N_{\text{secs}/\text{year}} \quad (1)$$

Each **K** and **V** matrix participates in one GEMV in each forward pass during decoding. For matrix **K**, it is the scoring computation. While for matrix **V**, it is the context computation. In GEMV, each matrix element participates in one multiplication and one addition, respectively. According to Table 2, this is 43 memsets for  $N_{\text{memsets}/\text{GEMV}}$ . For the estimation of  $N_{\text{GEMVs}/\text{sec}}$ , we assume a continuous 20 tokens/s decoding speed. According to these settings, the number of memsets on each cell is less than  $2.8 \times 10^{10}$  per year. It is noticeable that existing work has manufactured ReRAM with an endurance  $>10^{12}$  [36]. This means REPA will not suffer the endurance issue when using high-endurance ReRAM.

## 10 Related Work

**PIM for LLM acceleration.** PIM on the DRAM and emerging ReRAM memories are both popular research topics. Most DRAM PIM research places CMOS logic near memory banks to reduce data transfer overhead [15, 17–19, 53, 79]. Some of these solutions are GPU-free systems, which perform all operations by PIM [15, 17, 79]. Others are xPU-hybrid solutions overcoming the drawback of PIM by leveraging xPUs for computation-intensive tasks [18, 19, 53]. We also notice systems using reconfigurable computing [16, 41]. These solutions have minimized area overhead but lower performance, as wordlines within a DRAM cell array cannot be parallelized, which causes losses in parallelization. ReRAM PIM

research is mostly based on analog or reconfigurable computing. Analog ReRAM PIM is fast, but constrained by power and memory capacity. Existing studies use this paradigm for the fast process of sparse attention [26, 39, 44, 70]. However, their low capacity have prevented us from using them for KV cache offloading. Reconfigurable ReRAM PIM supports high capacity, with good performance in memory-intensive operations [24, 40, 73, 77]. Their edge over DRAM reconfigurable PIM is their compatibility to wordline parallelism, which exhibits a higher potential in parallelization.

**KV cache management and offloading.** The gap between the LLM size and GPU memory capacity has motivated research for the management and offloading of KV cache. Existing studies primarily focus on enhanced mechanisms and policies in this topic. From the perspective of mechanisms, they answer questions such as how GPU memory should be arranged [34, 58, 75], and how the offloading hierarchy is structured [13, 14, 59]. From the perspective of policies, existing studies endeavor to find better offloading timings [14, 22, 61, 78], and explore possibilities of sharing the offloaded KV cache in certain cases [13, 74]. KV cache compression and recomputation techniques are also discussed to further reduce the data size [11, 25, 43, 47, 72]. It is noticeable that our work is orthogonal to most of these studies. The REPA-PIM device can be integrated into the offloading system to provide offload-time KV cache processing, leaving both the original mechanisms and policies unchanged.

## 11 Conclusion

In this paper, we propose REPA, a GPU-PIM hybrid system for the joint acceleration of KV cache offloading and processing. Our system is characterized by high parallelism, which is achieved by in-depth optimizations on micro-architecture, data mapping and GPU-PIM pipelining. We propose bulk-wise memory instructions and multi-level controllers to enable fine-grained parallelism in the PIM device. To fully leverage this parallelization ability, we propose locality-aware data mapping, which significantly reduces the long-range data transfer overhead. At the system level, we adopt sub-batch pipelining to reduce the idleness inside a batch, and propose transfer overlapping to shadow the KV transfer overhead by computation. REPA is fast, efficient and highly integratable. It is 1.5–6.5× faster, and 6.2–6.3× more efficient than NVIDIA A100. When integrated into FlexGen, it achieves 1.2–1.4× inference speedup.

## Acknowledgments

We thank our shepherd Onur Mutlu, and the anonymous reviewers for their valuable insights and constructive feedback. This research was supported by the National Key Research & Development Program of China (No. 2022YFB4500103), NSFC (No. 62402317 and No. 62032008), and STCSM (No. 25LN3200900 and No. 24ZR1435500).

## References

- [1] Ameer Haj Ali, Rotem Ben Hur, Nimrod Wald, and Shahar Kvatinisky. 2018. Efficient Algorithms for In-Memory Fixed Point Multiplication Using MAGIC. In *IEEE International Symposium on Circuits and Systems, ISCAS 2018, 27–30 May 2018, Florence, Italy*. IEEE, 1–5. doi:10.1109/ISCAS.2018.8351561
- [2] Ankit Bende, Simranjeet Singh, Chandan Kumar Jha, Tim Kempen, Felix Cüppers, Christopher Bengel, Andre Zambanini, Dennis Nielinger, Sachin B. Patkar, Rolf Drechsler, Rainer Waser, Farhad Merchant, and Vikas Rana. 2024. Experimental Validation of Memristor-Aided Logic Using 1T1R TaOx RRAM Crossbar Array. In *37th International Conference on VLSI Design and 23rd International Conference on Embedded Systems, VLSID 2024, Kolkata, India, January 6–10, 2024*. IEEE, 565–570. doi:10.1109/VLSID60093.2024.00100
- [3] Mehrdad Biglari, Tobias Lieske, and Dietmar Fey. 2018. High-Endurance Bipolar ReRAM-Based Non-Volatile Flip-Flops with Run-Time Tunable Resistive States. In *Proceedings of the 14th IEEE/ACM International Symposium on Nanoscale Architectures, NANOARCH 2018, Athens, Greece, July 17–19, 2018*. ACM, 19–24. doi:10.1145/3232195.3232217
- [4] Baishakhi Rani Biswas, Claire Yuan, Fangzhou Wang, and Sandeep Gupta. 2024. Systematic Generation of Memristor-Transistor Single-Phase Combinational Logic Cells. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 43, 10 (2024), 2990–3003. doi:10.1109/TCAD.2024.3384012
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual*, Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [6] Junting Chen, Checheng Yu, Xunzhe Zhou, Tianqi Xu, Yao Mu, Mengkang Hu, Wenqi Shao, Yikai Wang, Guohao Li, and Lin Shao. 2025. EMOS: Embodiment-aware Heterogeneous Multi-robot Operating System with LLM Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=Ey8KcabBpB>
- [7] Pai-Yu Chen, Xiaochen Peng, and Shimeng Yu. 2018. NeuroSim: A Circuit-Level Macro Model for Benchmarking Neuro-Inspired Architectures in Online Learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 12 (2018), 3067–3080. doi:10.1109/TCAD.2018.2789723
- [8] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18–22, 2016*. IEEE Computer Society, 27–39. doi:10.1109/ISCA.2016.13
- [9] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi

- Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, and S. S. Li. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025). doi:10.48550/ARXIV.2501.12948 arXiv:2501.12948
- [10] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, and Wangding Zeng. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024). doi:10.48550/ARXIV.2412.19437 arXiv:2412.19437
- [11] Harry Dong, Xinyu Yang, Zhenyu Zhang, Zhangyang Wang, Yuejie Chi, and Beidi Chen. 2024. Get More with LESS: Synthesizing Recurrence with KV Cache Compression for Efficient LLM Inference. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=uHdHVKFMW>
- [12] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurélien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Rozière, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Grégoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel M. Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jency Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, and et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). doi:10.48550/ARXIV.2407.21783 arXiv:2407.21783
- [13] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, Saurabh Bagchi and Yiyang Zhang (Eds.). USENIX Association, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [14] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast State Restoration in LLM Serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*. ACM, 128–143. doi:10.1145/3689031.3696072
- [15] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi R. Iyer, and Reetuparna Das. 2025. PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 862–881. doi:10.1145/3676641.3716267
- [16] Nastaran Hajinazar, Geraldo F. Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri-Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, and Onur Mutlu. 2021. SIMDram: a framework for bit-serial SIMD processing using DRAM. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 329–345. doi:10.1145/3445814.3446749
- [17] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*. IEEE, 372–385. doi:10.1109/MICRO50266.2020.00040
- [18] Yintao He, Haiyu Mao, Christina Giannoula, Mohammad Sadrosadati, Juan Gómez-Luna, Huawei Li, Xiaowei Li, Ying Wang, and Onur Mutlu. 2025. PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 766–782. doi:10.1145/3676641.3716009
- [19] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024 - 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 722–737. doi:10.1145/3620666.3651380
- [20] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. *CoRR* abs/2406.17565 (2024). doi:10.48550/ARXIV.2406.17565 arXiv:2406.17565
- [21] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. *CoRR* abs/2401.11181 (2024). doi:10.48550/ARXIV.2401.11181 arXiv:2401.11181



- [22] Yitao Hu, Xiulong Liu, Guotao Yang, Linxuan Li, Kai Zeng, Zhixin Zhao, Sheng Chen, Laiping Zhao, Wenxin Li, and Keqiu Li. 2025. TightLLM: Maximizing Throughput for LLM Inference via Adaptive Offloading Policy. *IEEE Trans. Computers* 74, 7 (2025), 2195–2209. doi:10.1109/TC.2025.3558009
- [23] Rotem Ben Hur, Nimrod Wald, Nishil Talati, and Shahar Kvatin-sky. 2017. Simple magic: Synthesis and in-memory Mapping of logic execution for memristor-aided logic. In *2017 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2017, Irvine, CA, USA, November 13-16, 2017*, Sri Parameswaran (Ed.). IEEE, 225–232. doi:10.1109/ICCAD.2017.8203782
- [24] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: in-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 802–815. doi:10.1145/3307650.3322237
- [25] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annavaram. 2025. KVPR: Efficient LLM Inference with I/O-Aware KV Cache Partial Recomputation. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 19474–19488. doi:10.18653/v1/2025.findings-acl.997
- [26] Seungchul Jung, Hyungwoo Lee, Sungmeen Myung, Hyunsoo Kim, Seung Keun Yoon, Soon-Wan Kwon, Yongmin Ju, Minje Kim, Woosook Yi, Shinhee Han, et al. 2022. A crossbar array of magnetoresistive memory devices for in-memory computing. *Nature* 601, 7892 (2022), 211–216. doi:10.1038/s41586-021-04196-6
- [27] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 897–912. doi:10.1145/3676641.3715996
- [28] Akifumi Kawahara, Ken Kawai, Yuuichirou Ikeda, Yoshikazu Katoh, Ryotaro Azuma, Yuhei Yoshimoto, Kouhei Tanabe, Zhiqiang Wei, Takeki Ninomiya, Koji Katayama, Ryutaro Yasuhara, Shunsaku Muraoka, Atsushi Himeno, Naoki Yoshikawa, Hideaki Murase, Kazuhiko Shimakawa, Takeshi Takagi, Takumi Mikawa, and Kunitoshi Aono. 2013. Filament scaling forming technique and level-verify-write scheme with endurance over 107 cycles in ReRAM. In *2013 IEEE International Solid-State Circuits Conference - Digest of Technical Papers, ISSCC 2013, San Francisco, CA, USA, February 17-21, 2013*. IEEE, 220–221. doi:10.1109/ISSCC.2013.6487708
- [29] Joonyoung Kim and Younsu Kim. 2014. HBM: Memory solution for bandwidth-hungry processors. In *2014 IEEE Hot Chips 26 Symposium (HCS), Cupertino, CA, USA, August 10-12, 2014*. IEEE, 1–24. doi:10.1109/HOTCHIPS.2014.7478812
- [30] Tae-Hyun Kim, Byungkyu Song, In Jun Jung, and Seong-Ook Jung. 2022. A Sneak Current Compensation Scheme With Offset Cancellation Sensing Circuit for ReRAM-Based Cross-Point Memory Array. *IEEE Trans. Circuits Syst. I Regul. Pap.* 69, 4 (2022), 1583–1594. doi:10.1109/TCSI.2021.3133945
- [31] Jack Kosaian and K. V. Rashmi. 2021. Arithmetic-intensity-guided fault tolerance for neural network inference on GPUs. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, Bronis R. de Supinski, Mary W. Hall, and Todd Gamblin (Eds.). ACM, 79. doi:10.1145/3458817.3476184
- [32] Shahar Kvatin-sky, Dmitry Belousov, Slavik Liman, Guy Satat, Nimrod Wald, Eby G. Friedman, Avinoam Kolodny, and Uri C. Weiser. 2014. MAGIC - Memristor-Aided Logic. *IEEE Trans. Circuits Syst. II Express Briefs* 61-II, 11 (2014), 895–899. doi:10.1109/TCSII.2014.2357292
- [33] Shahar Kvatin-sky, Misbah Ramadan, Eby G. Friedman, and Avinoam Kolodny. 2015. VTEAM: A General Model for Voltage-Controlled Memristors. *IEEE Transactions on Circuits and Systems II: Express Briefs* 62, 8 (2015), 786–790. doi:10.1109/TCSII.2015.2433536
- [34] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600006.3613165
- [35] Jaeyong Lee, Hyeunjo Kim, Sanghun Oh, Myoungjun Chun, Myung-suk Kim, and Jihong Kim. 2025. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture, ISCA 2025, Tokyo, Japan, June 21-25, 2025*. ACM, 529–543. doi:10.1145/3695053.3731073
- [36] Myoung-Jae Lee, Chang Bum Lee, Dongsoo Lee, Seung Ryul Lee, Man Chang, Ji Hyun Hur, Young-Bae Kim, Chang-Jung Kim, David H. Seo, Sunae Seo, U-In Chung, In-Kyeong Yoo, and Kinam Kim. 2011. A Fast, High-Endurance and Scalable Non-Volatile Memory Device Made from Asymmetric Ta2O5-x/TaO2-x Bilayer Structures. *Nature Materials* 10, 8 (Aug. 2011), 625–630. doi:10.1038/nmat3070
- [37] Jan Leusmann, Anna Belardinelli, Luke Haliburton, Stephan Hasler, Albrecht Schmidt, Sven Mayer, Michael Gienger, and Chao Wang. 2025. Investigating LLM-Driven Curiosity in Human-Robot Interaction. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI 2025, Yokohama Japan, 26 April 2025- 1 May 2025*, Naomi Yamashita, Vanessa Evers, Koji Yatani, Sharon Xianghua Ding, Bongshin Lee, Marshini Chetty, and Phoebe O. Touns Dugas (Eds.). ACM, 599:1–599:16. doi:10.1145/3706598.3713923
- [38] Huize Li, Hai Jin, Long Zheng, and Xiaofei Liao. 2020. ReSQM: Accelerating Database Operations Using ReRAM-Based Content Addressable Memory. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 39, 11 (2020), 4030–4041. doi:10.1109/TCAD.2020.3012860
- [39] Huize Li, Hai Jin, Long Zheng, Xiaofei Liao, Yu Huang, Cong Liu, Jiahong Xu, Zhuohui Duan, Dan Chen, and Chuangyi Gui. 2024. CPSAA: Accelerating Sparse Attention Using Crossbar-Based Processing-In-Memory Architecture. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 43, 6 (2024), 1741–1754. doi:10.1109/TCAD.2023.3344524
- [40] Huize Li, Zhaoying Li, Zhenyu Bai, and Tulika Mitra. 2024. ASADI: Accelerating Sparse Attention Using Diagonal-based In-Situ Computing. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024, Edinburgh, United Kingdom, March 2-6, 2024*. IEEE, 774–787. doi:10.1109/HPCA57654.2024.00065
- [41] Shuangchen Li, Dimin Niu, Krishna T. Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. 2017. DRISA: a DRAM-based reconfigurable in-situ accelerator. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14-18, 2017*, Hillery C. Hunter, Jaime Moreno, Joel S. Emer, and Daniel Sánchez (Eds.). ACM, 288–301. doi:10.1145/3123939.3123977
- [42] Lizi Liao, Grace Hui Yang, and Chirag Shah. 2023. Proactive Conversational Agents in the Post-ChatGPT World. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*, Hsin-Hsi Chen, Wei-Jou (Edward) Duh, Hen-Hsen Huang, Makoto P. Kato, Josiane Mothe, and Barbara Poblete (Eds.). ACM, 3452–3455. doi:10.1145/3539618.3594250
- [43] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Reza Haffari, and Bohan Zhuang. 2024. MiniCache: KV Cache Compression in Depth



- Dimension for Large Language Models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). [http://papers.nips.cc/paper\\_files/paper/2024/hash/fd0705710bf01b88a60a3d479ea341d9-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2024/hash/fd0705710bf01b88a60a3d479ea341d9-Abstract-Conference.html)
- [44] Fangxin Liu, Wenbo Zhao, Zongwu Wang, Yongbiao Chen, Xiaoyao Liang, and Li Jiang. 2024. ERA-BS: Boosting the Efficiency of ReRAM-Based PIM Accelerator With Fine-Grained Bit-Level Sparsity. *IEEE Trans. Computers* 73, 9 (2024), 2320–2334. doi:10.1109/TC.2023.3290869
- [45] Mushui Liu, Yuhang Ma, Zhen Yang, Jun Dan, Yunlong Yu, Zeng Zhao, Zhipeng Hu, Bai Liu, and Changjie Fan. 2025. LLM4GEN: Leveraging Semantic Representation of LLMs for Text-to-Image Generation. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 5523–5531. doi:10.1609/AAAI.V39I5.32588
- [46] Tz-Yi Liu, Tian Hong Yan, Roy Scheuerlein, Yingchang Chen, Jeffrey KoonYee Lee, Gopinath Balakrishnan, Gordon Yee, Henry Zhang, Alex Yap, Jingwen Ouyang, Takahiko Sasaki, Sravanti Addepalli, Ali Al-Shamma, Chin-Yu Chen, Mayank Gupta, Greg Hilton, Saurabh Joshi, Achal Kathuria, Vincent Lai, Deep Masiwal, Masahide Matsumoto, Anurag Nigam, Anil Pai, Jayesh Pakhale, Chang Hua Siau, Xiaoxia Wu, Ronald Yin, Liping Peng, Jang Yong Kang, Sharon Huynh, Huijuan Wang, Nicolas Nagel, Yoichiro Tanaka, Masaaki Higashitani, Tim Minvielle, Chandu Gorla, Takayuki Tsukamoto, Takeshi Yamaguchi, Mutsumi Okajima, Takayuki Okamura, Satoru Takase, Takahiko Hara, Hirofumi Inoue, Luca Fasoli, Mehrdad Mofidi, Ritu Shrivastava, and Khandker Quader. 2013. A 130.7mm<sup>2</sup> 2-layer 32Gb ReRAM memory device in 24nm technology. In *2013 IEEE International Solid-State Circuits Conference - Digest of Technical Papers, ISSCC 2013, San Francisco, CA, USA, February 17-21, 2013*. IEEE, 210–211. doi:10.1109/ISSCC.2013.6487703
- [47] Yuhang Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*. ACM, 38–56. doi:10.1145/3651890.3672274
- [48] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyriillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). [http://papers.nips.cc/paper\\_files/paper/2023/hash/a452a7c6c463e4ae8fbd614c6e983e6-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2023/hash/a452a7c6c463e4ae8fbd614c6e983e6-Abstract-Conference.html)
- [49] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies, FAST 2021, February 23-25, 2021, Marcos K. Aguilera and Gala Yadgar (Eds.)*. USENIX Association, 203–216. <https://www.usenix.org/conference/fast21/presentation/mohan>
- [50] Shubham Negi, Utkarsh Saxena, Deepika Sharma, and Kaushik Roy. 2025. HCIM: ADC-Less Hybrid Analog-Digital Compute in Memory Accelerator for Deep Learning Workloads. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (Tokyo, Japan) (ASPAC '25)*. Association for Computing Machinery, New York, NY, USA, 648–655. doi:10.1145/3658617.3697572
- [51] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1-5, 2025*. IEEE, 1510–1525. doi:10.1109/HPCA61900.2025.00113
- [52] Artemis Panagopoulou, Le Xue, Ning Yu, Junnan Li, Dongxu Li, Shafiq Joty, Ran Xu, Silvio Savarese, Caiming Xiong, and Juan Carlos Niebles. 2024. X-InstructBLIP: A Framework for Aligning Image, 3D, Audio, Video to LLMs and its Emergent Cross-Modal Reasoning. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XLV (Lecture Notes in Computer Science, Vol. 15103)*, Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Springer, 177–197. doi:10.1007/978-3-031-72995-9\_11
- [53] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024 - 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 103–119. doi:10.1145/3620665.3640422
- [54] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 8024–8035. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [55] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [56] Xiaochen Peng, Wriddhi Chakraborty, Ankit Kaul, Wonbo Shim, Muhammad S Bakir, Suman Datta, and Shimeng Yu. 2020. Benchmarking Monolithic 3D Integration for Compute-in-Memory Accelerators: Overcoming ADC Bottlenecks and Maintaining Scalability to 7nm or Beyond. In *2020 IEEE International Electron Devices Meeting (IEDM)*. 30.4.1–30.4.4. doi:10.1109/IEDM13553.2020.9372091
- [57] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*, Dawn Song, Michael Carbin, and Tianqi Chen (Eds.). mlsys.org. [https://proceedings.mlsys.org/paper\\_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html](https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html)
- [58] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 1133–1150. doi:10.1145/3669940.3707256
- [59] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake:

- Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25–27, 2025*, Haryadi S. Gunawi and Vasily Tarasov (Eds.). USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [60] Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2025. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=moXtEmCleY>
- [61] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *International Conference on Machine Learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 31094–31116. <https://proceedings.mlr.press/v202/sheng23a.html>
- [62] Devesh Singh and Donald Yeung. 2024. MORSE: Memory Overwrite Time Guided Soft Writes to Improve ReRAM Energy and Endurance. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques, PACT 2024, Long Beach, CA, USA, October 14–16, 2024*. ACM, 26–39. doi:10.1145/3656019.3676890
- [63] Dmitri B Strukov, Gregory S Snider, Duncan R Stewart, and R Stanley Williams. 2008. The missing memristor found. *Nature* 453, 7191 (2008), 80–83. doi:10.1038/nature06932
- [64] Xiaotian Sun, Xinyu Wang, Wanqian Li, Yinhe Han, and Xiaoming Chen. 2025. PIMCOMP: An End-to-End DNN Compiler for Processing-In-Memory Accelerators. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 44, 5 (2025), 1745–1759. doi:10.1109/TCAD.2024.3496847
- [65] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xi-ang Kuan, Puxin Xu, Zheng Yan, Illyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR abs/2307.09288* (2023). doi:10.48550/ARXIV.2307.09288 arXiv:2307.09288
- [66] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [67] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. 2023. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23–26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 364–381. doi:10.1145/3600006.3613145
- [68] Lei Xie, Hoang Anh Du Nguyen, Mottaqiallah Taouil, Said Hamdioui, and Koen Bertels. 2018. A Mapping Methodology of Boolean Logic Circuits on Memristor Crossbar. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 37, 2 (2018), 311–323. doi:10.1109/TCAD.2017.2695880
- [69] Cong Xu, Dimin Niu, Naveen Muralimanohar, Rajeev Balasubramanian, Tao Zhang, Shimeng Yu, and Yuan Xie. 2015. Overcoming the challenges of crossbar resistive memory architectures. In *21st IEEE International Symposium on High Performance Computer Architecture, HPCA 2015, Burlingame, CA, USA, February 7–11, 2015*. IEEE Computer Society, 476–488. doi:10.1109/HPCA.2015.7056056
- [70] Jiahong Xu, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xiaokang Yang, Huize Li, Cong Liu, Fubing Mao, and Yu Zhang. 2024. ReHarvest: An ADC Resource-Harvesting Crossbar Architecture for ReRAM-Based DNN Accelerators. *ACM Trans. Archit. Code Optim.* 21, 3 (2024), 63:1–63:26. doi:10.1145/3659208
- [71] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *CoRR abs/2412.15115* (2024). doi:10.48550/ARXIV.2412.15115 arXiv:2412.15115
- [72] Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11–16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 3258–3270. doi:10.18653/V1/2024.FINDINGS-ACL.195
- [73] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. 2020. ReTransformer: ReRAM-based Processing-in-Memory Architecture for Transformer Acceleration. In *IEEE/ACM International Conference On Computer Aided Design, ICCAD 2020, San Diego, CA, USA, November 2–5, 2020*. IEEE, 92:1–92:9. doi:10.1145/3400302.3415640
- [74] Yifei Yang, Zouying Cao, Qiguang Chen, Libo Qin, Dongjie Yang, Hai Zhao, and Zhi Chen. 2024. KVSharer: Efficient Inference via Layer-Wise Dissimilar KV Cache Sharing. *CoRR abs/2410.18517* (2024). doi:10.48550/ARXIV.2410.18517 arXiv:2410.18517
- [75] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11–16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 11608–11620. doi:10.18653/V1/2024.ACL-LONG.623
- [76] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11–13, 2022*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/you>
- [77] Geng Yuan, Payman Behnam, Zhengang Li, Ali Shafiee, Sheng Lin, Xiaolong Ma, Hang Liu, Xuehai Qian, Mahdi Nazm Bojnordi, Yanzhi Wang, and Caiwen Ding. 2021. FORMS: Fine-grained Polarized ReRAM-based In-situ Computation for Mixed-signal DNN Accelerator. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14–18, 2021*. IEEE, 265–278. doi:10.1109/ISCA52012.2021.00029

- [78] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). [http://papers.nips.cc/paper\\_files/paper/2023/hash/6ceefa7b15572587b78ecfceb2827f8-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfceb2827f8-Abstract-Conference.html)
- [79] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 1071–1085. doi:10.1109/HPCA53966.2022.00082